

Lab 1 Report

Huzaifa Iqbal 31563

Executive Summary:

The performance testing reveals that while the **LinkedListStack demonstrates faster single-operation execution**, the **ArrayStack provides better sustained performance** over multiple iterations, making it more efficient for bulk operations.

Detailed Results:

Single Iteration Performance:

ArrayStack: 4.139×10^{-5} seconds (41.39 microseconds)

LinkedListStack: 1.679×10^{-5} seconds (16.79 microseconds)

LinkedListStack was 59.4% faster in single execution

Average Performance (1000 iterations):

ArrayStack: 7.713×10^{-7} seconds (0.771 microseconds)

LinkedListStack: 9.527×10^{-7} seconds (0.953 microseconds)

ArrayStack was 19.0% faster on average

Technical Analysis:

Single Operation Advantage: LinkedListStack's $O(1)$ insertion and deletion operations at the head provide significant speed advantages for individual operations, as evidenced by the 59.4% faster single-run performance.

Bulk Operation Efficiency: ArrayStack's superior performance in the 1000-iteration test demonstrates better cache locality and memory access patterns.

Arrays provide contiguous memory storage, reducing cache misses during sustained operations.

Memory Overhead: LinkedListStack requires additional memory for node pointers (typically 8-16 bytes per element), while ArrayStack has minimal overhead beyond the actual data storage.

Performance Consistency: The smaller standard deviation in ArrayStack's performance (evident from the better average) indicates more predictable execution times, which is crucial for real-time text editing applications.

Conclusion:

The LinkedListStack excels in scenarios requiring optimal single-operation performance, while the ArrayStack proves more efficient for applications involving sustained, multiple operations. For text editing applications where users typically perform sequences of operations (typing, deleting, cursor movement), the ArrayStack implementation is recommended due to its better overall performance in realistic usage patterns.